Logical Volume Management (LVM)

Flexible disk storage in Linux: physical volumes, volume groups and logical volumes - create, format, mount and resize, with examples, a diagram, command chart and interview Q&A.

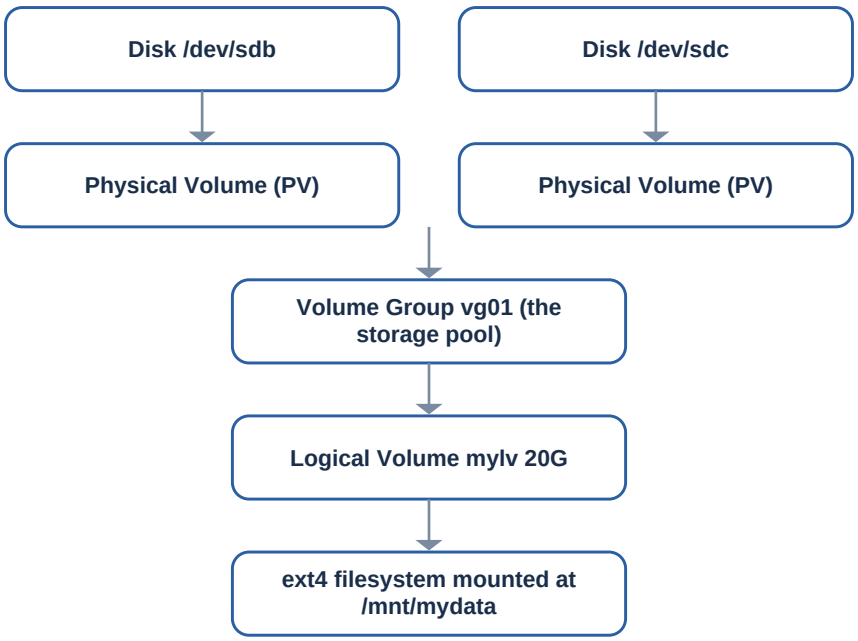
ROOT = changes the system or reads protected data; run it from a root shell. **ANY USER** = read-only, safe for everyone.

How to read this: every command here changes the system or reads protected disk metadata, so examples are ROOT. Practice on disposable lab disks and keep backups - pvcreate, mkfs and resize operations can destroy data. Replace /dev/sdb, vg01 and mylv with your real names.

1. What LVM is

Logical Volume Management is a layer between your disks and your filesystems. Instead of formatting a fixed partition, you pool disks together and carve out **logical volumes** that you can **extend later without downtime** and without repartitioning. (Shrinking is possible too, but it is the risky, filesystem-dependent direction - section 7.) It is the standard way to manage storage flexibly on Linux servers.

LVM stacks three layers:



Physical volumes are the raw disks or partitions LVM manages. A **volume group** pools several PVs into one bucket of space. **Logical volumes** are slices of that pool - this is what you format and mount.

2. Step 1 - Create physical volumes

```
# ROOT - turn a raw disk into a physical volume
pvcreate /dev/sdb /dev/sdc
# ROOT - verify the PVs were created (reads disk metadata)
pvdisplay
```

Replace /dev/sdb with your actual disk identifier. You can list available disks first with lsblk.

3. Step 2 - Create a volume group

```
# ROOT - pool one or more PVs into a volume group
vgcreate <VG_Name> /dev/sdb /dev/sdc
vgcreate vg01 /dev/sdb /dev/sdc
# ROOT - check the group's status and free space
vgdisplay
```

The volume group now holds the combined space of every PV you gave it. Add more disks later with vgextend.

4. Step 3 - Create logical volumes

```
# ROOT - carve a 20G logical volume named mylv out of vg01
lvcreate -L 20G -n mylv vg01
# ROOT - list logical volumes
lvdisplay
```

`-L` sets the size, `-n` the name. The LV appears as a device at `/dev/vg01/mylv`.

5. Step 4 - Put a filesystem on it

```
# ROOT - format the LV as ext4 (xfs is another common choice)
mkfs.ext4 /dev/vg01/mylv
```

6. Step 5 - Mount the logical volume

```
# ROOT - create the mount point if it does not exist yet
mkdir -p /mnt/mydata
# ROOT
mount /dev/vg01/mylv /mnt/mydata
```

The LV is now usable storage. As with plain partitions, add it to `/etc/fstab` to mount automatically at boot.

7. Resizing - the reason LVM exists

Need more space on mylv? Extend it from free space in the volume group, then grow the filesystem to match - all while it stays mounted. Growing is the lower-risk direction and works online - but lower risk is not risk-free, so keep backups:

```
# ROOT - add 10G to the logical volume
lvextend -L +10G /dev/vg01/mylv
# ROOT - grow the filesystem into the new space
# ext4:
resize2fs /dev/vg01/mylv
# xfs (grown with the filesystem mounted, aimed at the mount point):
xfs_growfs /mnt/mydata
```

Remember: `lvextend` grows the container, `resize2fs` or `xfs_growfs` grows the filesystem inside it - you need both, and both work without unmounting.

Shrinking is a different story, and it depends on the filesystem. **XFS cannot be shrunk at all** - only grown. **ext4 can be shrunk, but never while mounted**. The safe order for ext4 is: back up, unmount, check the filesystem, shrink the filesystem first, then reduce the LV, and remount:

```
# ROOT - ext4 shrink, in this exact order, with a backup first
umount /mnt/mydata
e2fsck -f /dev/vg01/mylv
resize2fs /dev/vg01/mylv 9G     # shrink the FILESYSTEM first
lvreduce -L 9G /dev/vg01/mylv   # only then reduce the volume
mount /dev/vg01/mylv /mnt/mydata
```

The filesystem must be shrunk to its new size **before** the volume is reduced, and never below the size of your data - get the order wrong and the filesystem is destroyed. Extending is routine; reducing deserves a backup.

Command chart

Command	What it does	Example
`pvcreate`	Turn a disk/partition into a physical volume (ROOT)	`pvcreate /dev/sdb`
`pvdisplay`	Show physical volumes (ROOT)	`pvdisplay`
`vgcreate`	Pool PVs into a volume group (ROOT)	`vgcreate vg01 /dev/sdb`
`vgdisplay`	Show volume groups and free space (ROOT)	`vgdisplay`
`lvcreate`	Create a logical volume (ROOT)	`lvcreate -L 20G -n mylv vg01`
`lvdisplay`	Show logical volumes (ROOT)	`lvdisplay`
`mkfs.ext4`	Format an LV (ROOT)	`mkfs.ext4 /dev/vg01/mylv`
`mount`	Mount an LV (ROOT)	`mount /dev/vg01/mylv /mnt/mydata`
`lvextend`	Grow an LV (ROOT)	`lvextend -L +10G /dev/vg01/mylv`
`resize2fs / xfs_growfs`	Grow the filesystem after lvextend, online (ROOT)	`resize2fs /dev/vg01/mylv`
`lvreduce`	Shrink an LV: filesystem-dependent - offline for ext4, filesystem first; XFS cannot shrink (ROOT)	`lvreduce -L 9G /dev/vg01/mylv`

Interview Q&A

Q. What is LVM?

A storage management layer that pools disks into volume groups and hands out resizable logical volumes, instead of fixed partitions.

Q. Name the three LVM layers.

Physical volumes (the raw disks), a volume group (the pool of their space), and logical volumes (the slices you format and mount).

Q. Why use LVM instead of plain partitions?

Flexibility: logical volumes can be extended without downtime or repartitioning, and a volume can span several disks.

Q. Which commands create the three layers?

pvcreate makes physical volumes, vgcreate pools them into a volume group, lvcreate carves out a logical volume.

Q. You ran lvextend but df shows the old size. What is missing?

The filesystem was not grown. After lvextend, run resize2fs for ext4 or xfs_growfs for XFS - the LV is bigger, but the filesystem needs its own grow step.

Q. Is lvreduce as safe as lvextend?

No. Extending is the lower-risk, online direction - still with backups. Shrinking is offline for ext4 (unmount, e2fsck, resize2fs smaller, then lvreduce) and impossible for XFS, which can only grow. Always back up before reducing.

Sources: class slides (Logical Volume Management) and the Linux man pages (man lvm, man pvcreate, man vgcreate, man lvcreate, man lvextend, man resize2fs).